

Michael Nuttall,
Colin Duckworth, Colten Ross

Evaluating Mushroom Edibility

Spring 2026

Michael

Mushrooms are delicious, but unfortunately many are not edible, or as the old adage, goes, “All mushrooms are edible, but some you can only eat once.” So we made it our goal to use machine learning to classify mushrooms as either edible or poisonous/inedible using traits that a mycologist might observe, and to help prevent poisoning from eating toxic mushrooms.

Since we’re trying to assign each mushroom into one of two classes: edible or poisonous, this is a binary classification problem, which informed our choice of models throughout the project.

Machine learning classification algorithms on a mushroom dataset.

Dataset contains mushrooms physical descriptions as 22 features correlating to a classification of edible or poisonous.

This dataset contains samples corresponding to 23 species of gilled mushrooms drawn from The Audubon Society Field Guide to North American Mushrooms (1981).

Spring 2026

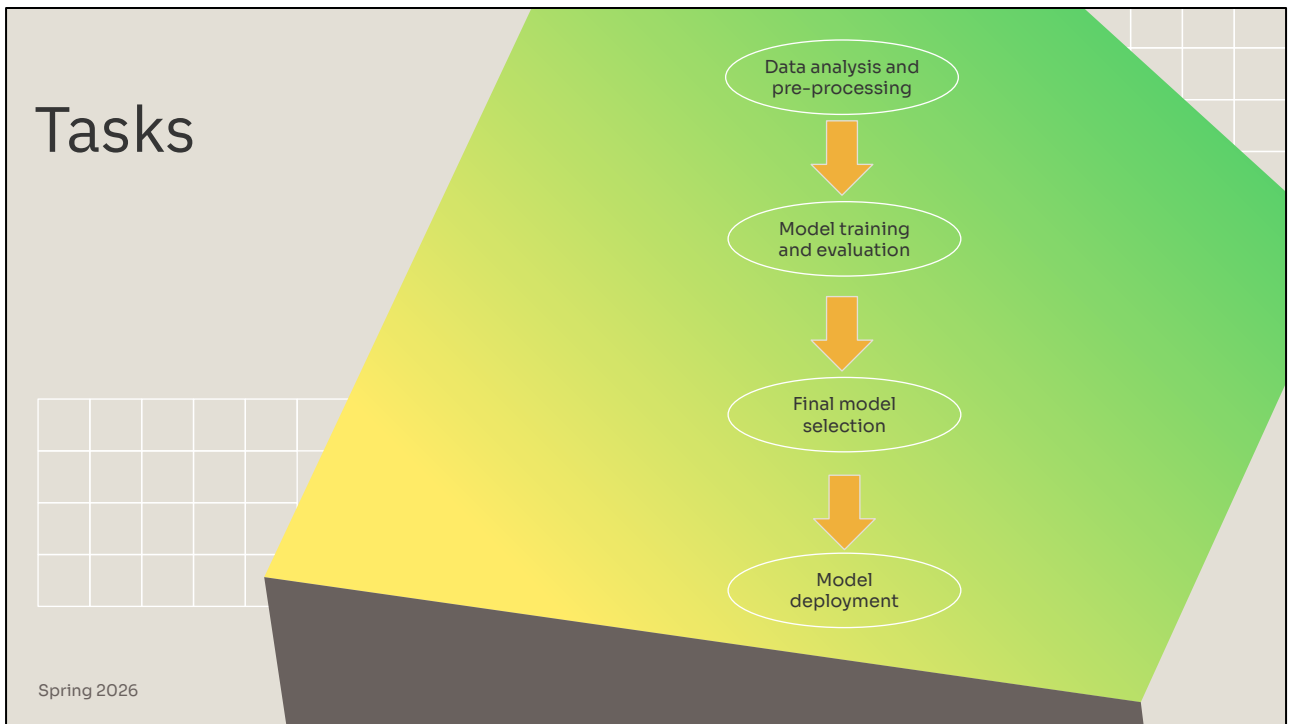
Colin

The problem was pretty straightforward: given a set of mushroom traits, can we predict whether that mushroom is edible or poisonous?

Our dataset came from the Kaggle Mushroom Classification dataset, which is based on the UCI mushroom dataset.

It represents 23 species of gilled mushrooms in the Agaricus and Lepiota family, containing 8,124 samples and 22 features including attributes such as odor, spore color, ring type, and other physical characteristics. Each mushroom is also labeled as either edible or poisonous. The original dataset also had mushrooms of unknown edibility, but those were grouped into the poisonous or not-recommended category.

We should note that this model is only learning from this limited dataset and fails to consider every mushroom in North America, let alone the world, so this should not be used as a general mushroom identification tool.



Colin

The important thing to note is that this is all categorical data. We are not dealing with numerical measurements like height or weight, but rather categories like odor being foul or absent, gill size being broad or narrow, and so on.

Because of that, traditional scaling or normalization was not really necessary, since there are no continuous numeric features where one variable might be on a much bigger scale than another.

Data Pre-Processing

	Potential challenge	How we responded
1	Missing or incomplete entries in the dataset.	We replaced missing and unknown data entries with the mean value of the column for the row index missing data.
2	Non numerical data values.	All of the data was encoded using the one hot encoder method to transfer the data from letter categories into numerical values for the model.
3	Scaling of the encoded data.	The data was purely categorical so the numerical values required no scaling to allow proper model behavior.

Spring 2026

Michael

The dataset was relatively clean. The main cleanup issue was that one feature, `stalk-root`, had missing values represented by a question mark.

Instead of eliminating these rows, we treated the question marks as missing values. If `stalk-root` was being used by a model, we imputed those missing values by replacing them with the predicted most common value from the training data.

But as it turned out, `stalk-root` was not one of the most important features in our final models, so for the improved top-feature models, this missing-value issue mostly disappeared since that column was not included.

That said, a lack of data can itself be data, so one future improvement could be to explicitly label missing values as their own category to test whether the missingness itself is predictive.

For the rest of the data, the main preprocessing step was one-hot encoding.

Most of the features had multiple possible categories and so were not binary. So instead of saying a mushroom cap is red, white, or brown in one column, we created separate boolean columns; for example, `cap_color_red`, `white`, and `brown` as being true or false.

Model evaluation and testing

How we'll compare different models

Step 1

Create a modularized setup which breaks data processing, training, and evaluation into separate steps.

Step 2

Use the pre-processed data to train various different models and compare them using the same metrics.

Step 3

Identify the top performing/most accurate models for further evaluation.

Spring 2026

Colten

We then began training a wide variety of model types, starting with a basic Random Forest model by Colin, and then expanded from there, including a dummy classifier, and then Logistic Regression, Decision Tree, and Random Forest classification models, chosen since they work well with structured tabular data and can capture interactions between features.

We also added XGBoost, a sequential variant of Random Forest.

A unique aspect of our project was that we took a modular approach, designed by Michael.

Instead of writing a totally separate program for every model, we separated the project into reusable pieces, including preprocessing, feature engineering, model training, evaluation, plots, and outputs.

experiment_name	model_class	accuracy	f1	cv_accuracy_mean	cv_f1_mean	
decision_tree_split_70_30_cv3	DecisionTreeClassifier	1.000000	1.000000	0.999631	0.999617	[8] Running decision_tree_top3_features ...
decision_tree_split_70_30_cv5	DecisionTreeClassifier	1.000000	1.000000	0.999631	0.999616	[9] Running decision_tree_top5_features ...
decision_tree_top7_features	DecisionTreeClassifier	1.000000	1.000000	1.000000	1.000000	[10] Running decision_tree_top7_features ...
decision_tree_top8_features	DecisionTreeClassifier	1.000000	1.000000	1.000000	1.000000	[11] Running decision_tree_top8_features ...
logreg_engineered	LogisticRegression	1.000000	1.000000	1.000000	1.000000	[12] Running decision_tree_topall_features ...
logreg_engineered_comparison	LogisticRegression	1.000000	1.000000	1.000000	1.000000	[13] Running random_forest_top1_features ...
random_forest_baseline	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[14] Running random_forest_top3_features ...
random_forest_engineered_comparison	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[15] Running random_forest_top5_features ...
test_no_engineering_comparison	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[16] Running random_forest_top7_features ...
random_forest_split_30	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[17] Running random_forest_top8_features ...
random_forest_split_70_30_cv3	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[18] Running random_forest_topall_features ...
random_forest_split_70_30_cv5	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[1] Running logreg_no_engineering_comparison ...
random_forest_split_80_20_cv3	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[2] Running logreg_engineered_comparison ...
random_forest_split_80_20_cv5	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[3] Running decision_tree_no_engineering_comparison ...
random_forest_top7_features	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[4] Running decision_tree_engineered_comparison ...
random_forest_top8	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[5] Running random_forest_no_engineering_comparison ...
random_forest_top8_features	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	[6] Running random_forest_engineered_comparison ...
random_forest_topall_features	RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	
xgboost_baseline	XGBClassifier	1.000000	1.000000	1.000000	1.000000	Top-k sweep summary:
xgboost_top8	XGBClassifier	1.000000	1.000000	1.000000	1.000000	experiment_name
logreg_split_70_30_cv3	LogisticRegression	0.998769	0.998722	0.999754	0.999744	model_class
logreg_split_70_30_cv5	LogisticRegression	0.998769	0.998722	0.999877	0.999872	top_n_features
decision_tree_baseline	DecisionTreeClassifier	0.998769	0.998721	0.999631	0.999616	accuracy
on_tree_engineered_comparison	DecisionTreeClassifier	0.998769	0.998721	0.999508	0.999489	decision_tree_top1_features
ree_no_engineering_comparison	DecisionTreeClassifier	0.998769	0.998721	0.999631	0.999616	decision_tree_top3_features
decision_tree_split_80_20_cv3	DecisionTreeClassifier	0.998769	0.998721	0.999631	0.999617	decision_tree_top5_features
decision_tree_split_80_20_cv5	DecisionTreeClassifier	0.998769	0.998721	0.999631	0.999616	decision_tree_top7_features
decision_tree_topall_features	DecisionTreeClassifier	0.998769	0.998721	0.999631	0.999616	decision_tree_top8_features
logreg_baseline	LogisticRegression	0.998769	0.998721	0.999877	0.999872	decision_tree_topall_features
reg_no_engineering_comparison	LogisticRegression	0.998769	0.998721	0.999877	0.999872	logreg_top1_features
logreg_split_80_20_cv3	LogisticRegression	0.998769	0.998721	0.999877	0.999872	logreg_top3_features
logreg_split_80_20_cv5	LogisticRegression	0.998769	0.998721	0.999877	0.999872	logreg_top5_features
logreg_topall_features	LogisticRegression	0.998769	0.998721	0.999877	0.999872	logreg_top7_features
decision_tree_top5_features	DecisionTreeClassifier	0.998154	0.998081	0.999015	0.998977	logreg_top8_features
logreg_top7_features	LogisticRegression	0.998154	0.998081	0.999015	0.998977	logreg_topall_features
logreg_top8_features	LogisticRegression	0.998154	0.998081	0.999015	0.998977	random_forest_top1_features
random_forest_top5_features	RandomForestClassifier	0.998154	0.998081	0.999015	0.998977	random_forest_top3_features
logreg_top5_features	LogisticRegression	0.996923	0.996797	0.998277	0.998208	random_forest_top5_features
decision_tree_top3_features	DecisionTreeClassifier	0.993231	0.992926	0.994584	0.994348	random_forest_top7_features
random_forest_top3_features	RandomForestClassifier	0.993231	0.992926	0.994584	0.994348	random_forest_top8_features
logreg_top3_features	LogisticRegression	0.992615	0.992278	0.994091	0.993831	random_forest_topall_features
decision_tree_top1_features	DecisionTreeClassifier	0.985846	0.985094	0.985229	0.984431	
logreg_top1_features	LogisticRegression	0.985846	0.985094	0.985229	0.984431	
random_forest_top1_features	RandomForestClassifier	0.985846	0.985094	0.985229	0.984431	
dummy_baseline	DummyClassifier	0.518154	0.000000	0.517971	0.000000	

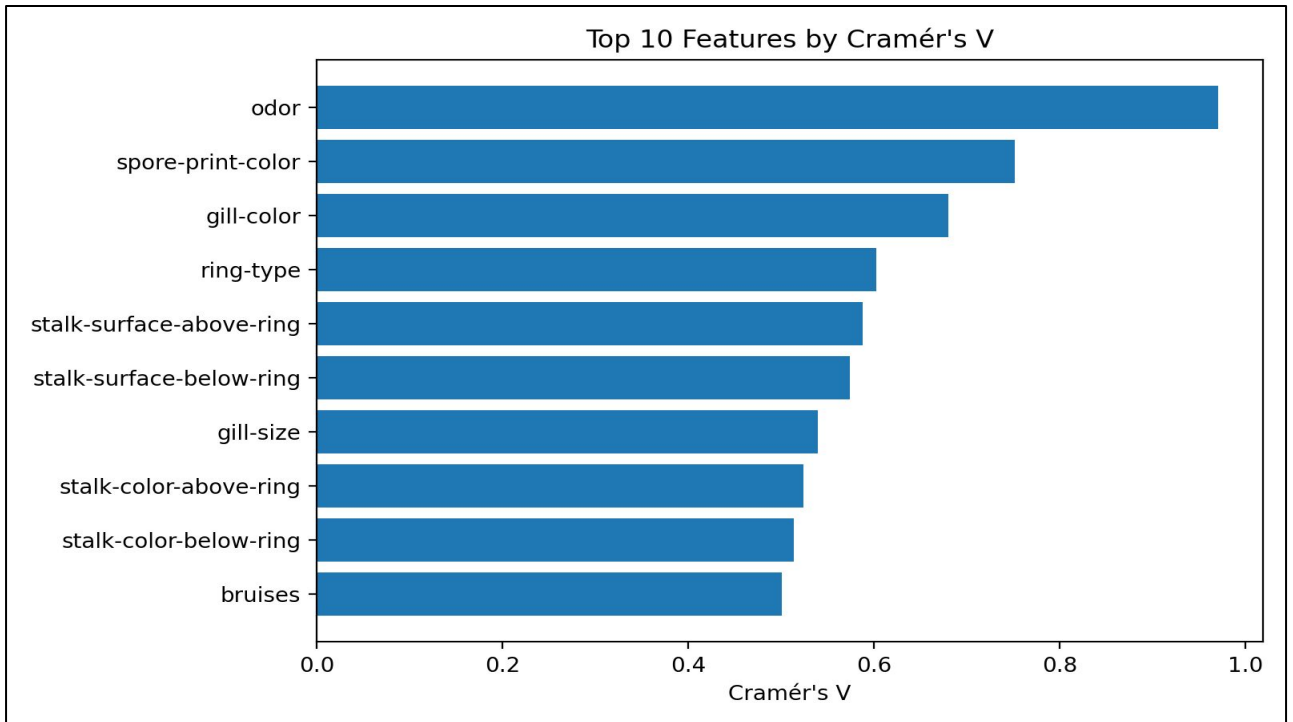
Michael

This allowed us to easily configure new experiments, specifying which model, what split, and how many folds to use, whether to use top features, save SHAP plots, and so on.

That let us test dozens of models at a time, over 40 model variants in total, without manually rewriting everything, saving ourselves, the programmers, a lot of time.

To compare models fairly, we used a consistent process, using the same random seed (of 42, the best number) so the results were repeatable and comparable.

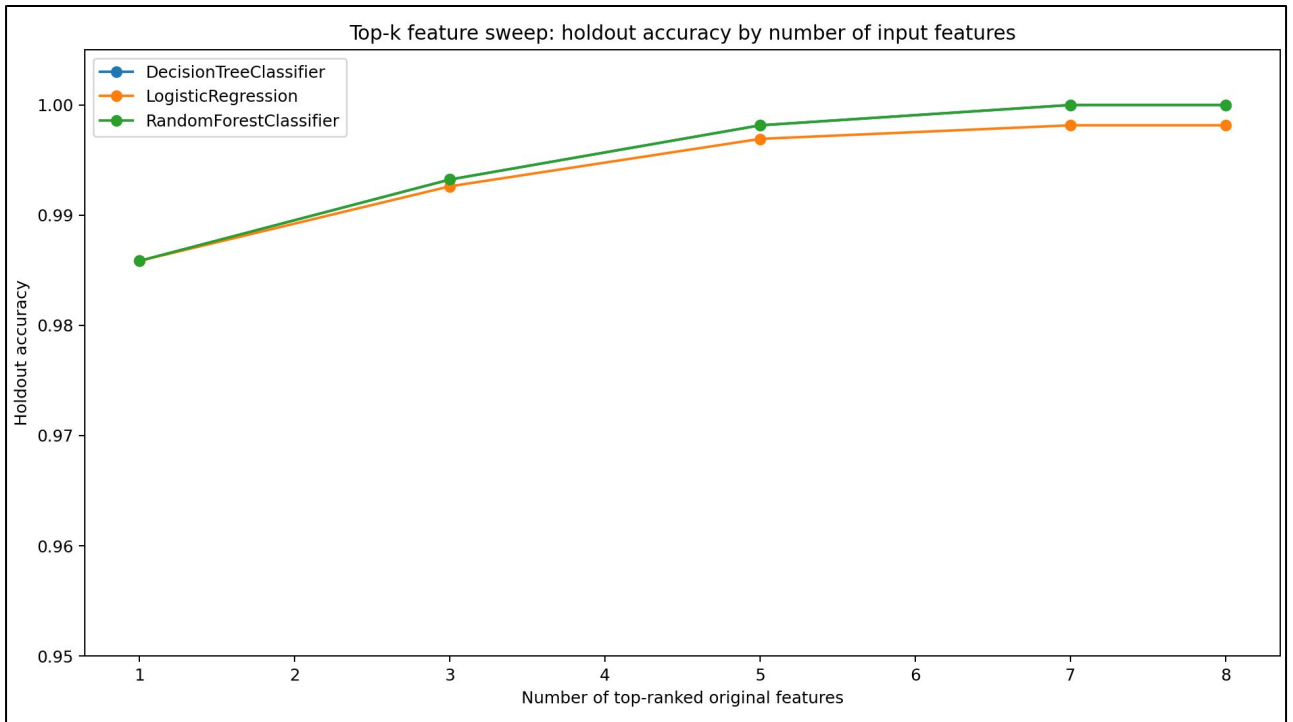
In each sweep, we tested various stratified train-test splits (such as 80/20 and 70/30), and also used cross-validation, such as 3- and 5-fold.



Colten

We also used feature selection engineering.

Since the data is categorical, we used Cramér's V to estimate how strongly each feature was associated with the target class. Then we tested models using different numbers of top-ranked features from one to all features.



Michael

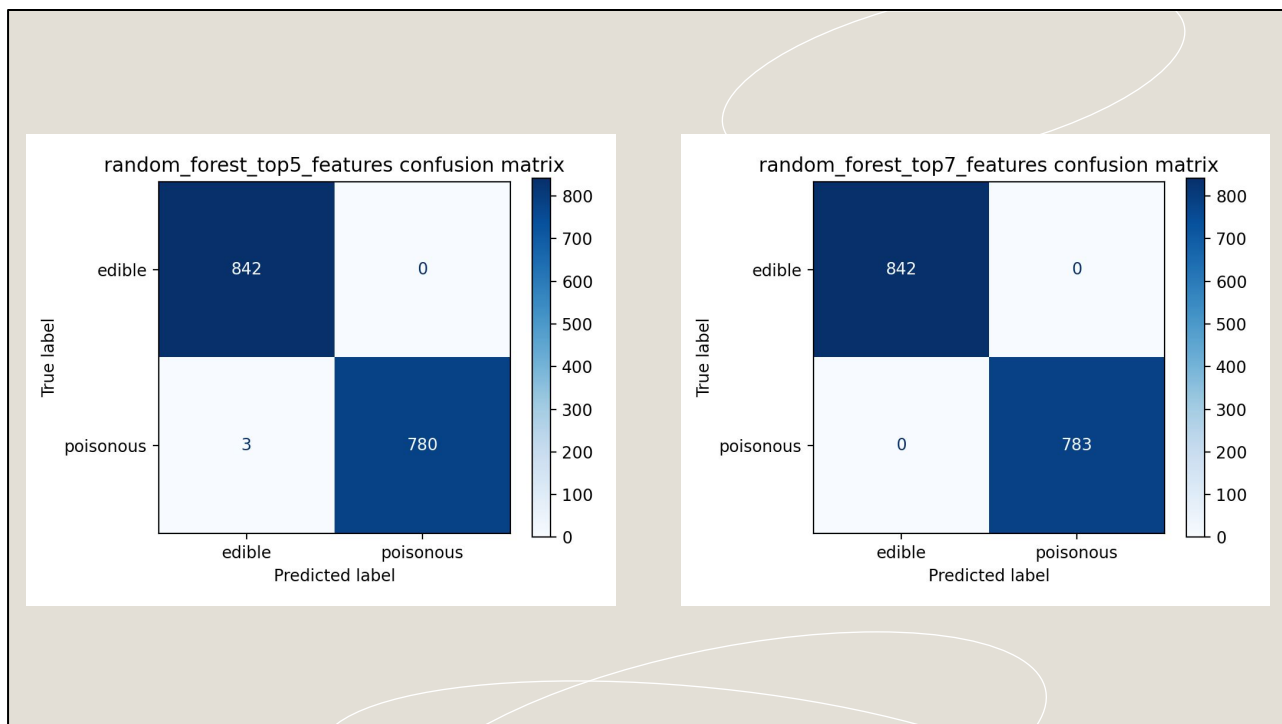
As it turns out, it wasn't difficult to find models that performed extremely well.

Some models achieved 100% accuracy and F1 score, including Logistic Regression with engineered features, Decision Tree, Random Forest, and XGBoost, informing us that the features in the dataset have a very strong relationship to the edible or poisonous label.

Thus, the interesting question became less 'Can we get high accuracy?' and more 'Can we get high accuracy with fewer features and explain the result?'

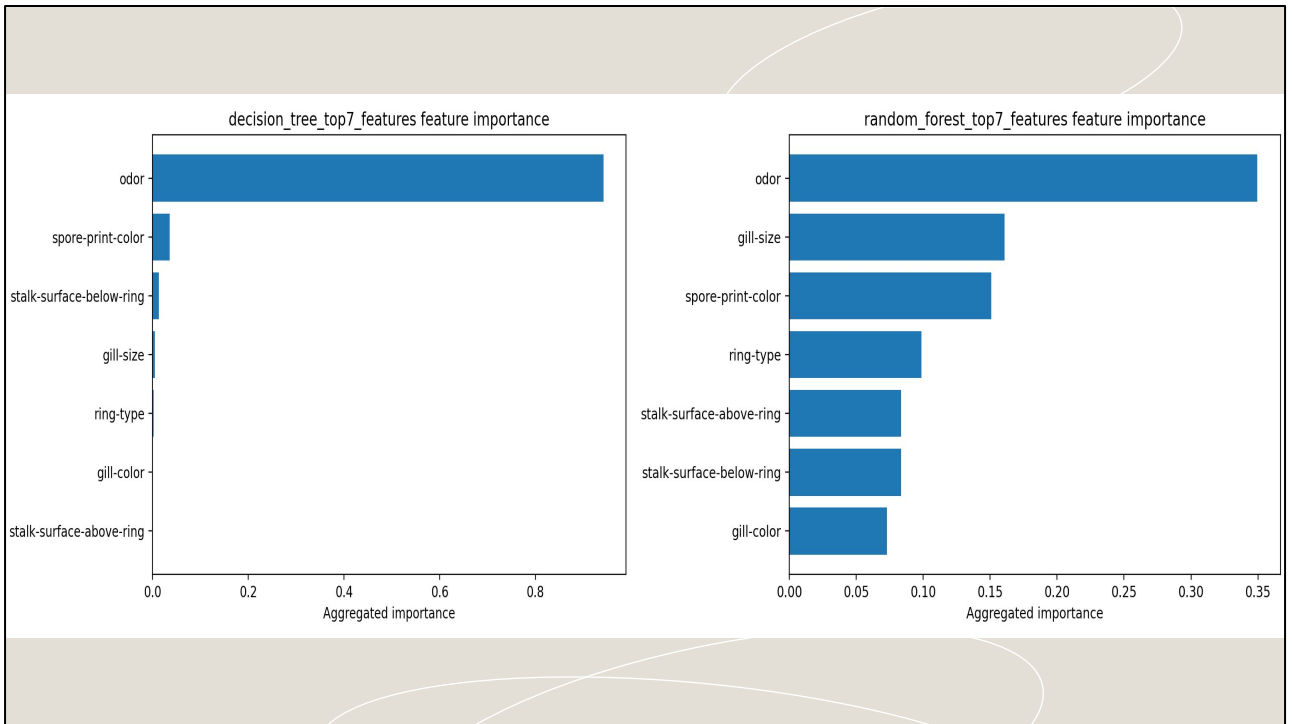
We also tested feature engineering, including domain-inspired features and interactions, considering bruising and habitat, but the strongest final models ended up working extremely well with just the top original features.

Out of the models we tested, two models were able to produce 100% accuracy and F1 score using only seven of the most significant features: a Decision Tree and a Random Forest, both with an 80/20 stratified split and 5-fold cross-validation.



Colin

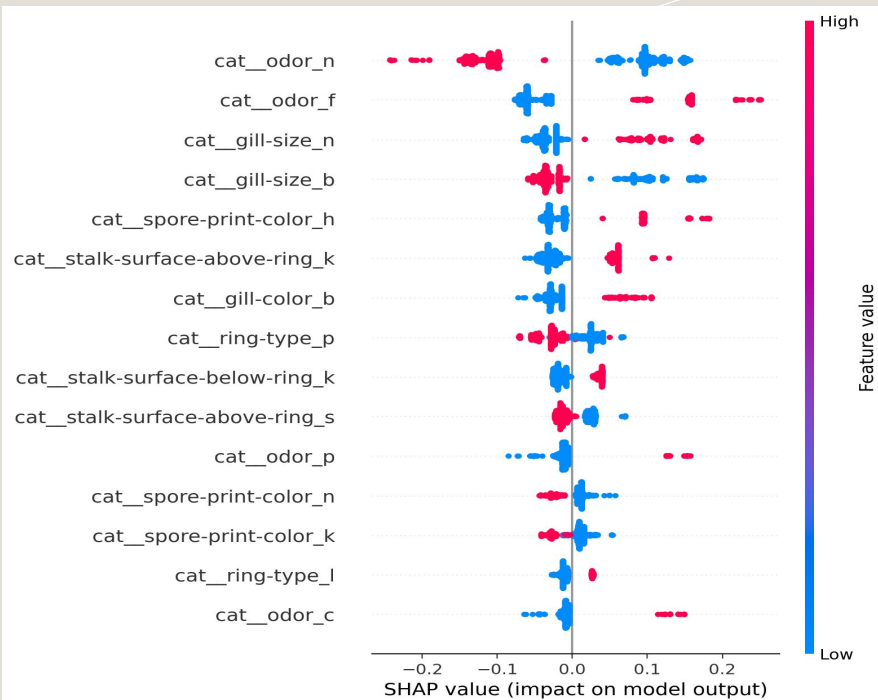
The top 5 feature versions were close, but not perfect. In fact, they misclassified poisonous mushrooms as edible. The top 7 versions were the smallest feature count we tested that reached perfect holdout accuracy and perfect cross-validation scores for both Decision Tree and Random Forest.



Colin

Interestingly, both the Decision Tree and Random Forest used the same seven selected features. Their relative importance varied somewhat between models, but the set of features was the same. This is logical since Random Forest is essentially an ensemble of decision trees and uses the same underlying idea of splitting data based on features.

Across the models, odor was consistently the most important feature.



Colten

The Random Forest is a more robust ensemble model suitable for our application, but it's less explainable. Instead, we must rely more on feature importance and SHAP (SHapley Additive exPlanations)-style explanations.

Model Deployment

Tkinter was used to build the application, so users can interact with the model without needing to use the command line.

We used mapped values to allow users to use regular words while the application converts them into the values expected by the model. To connect with the model, the interface used Pickle as well as Joblib to save and reuse the model after training was complete. This way instead of training the model every time the program starts we can train once, export to a file and load it later instantly.

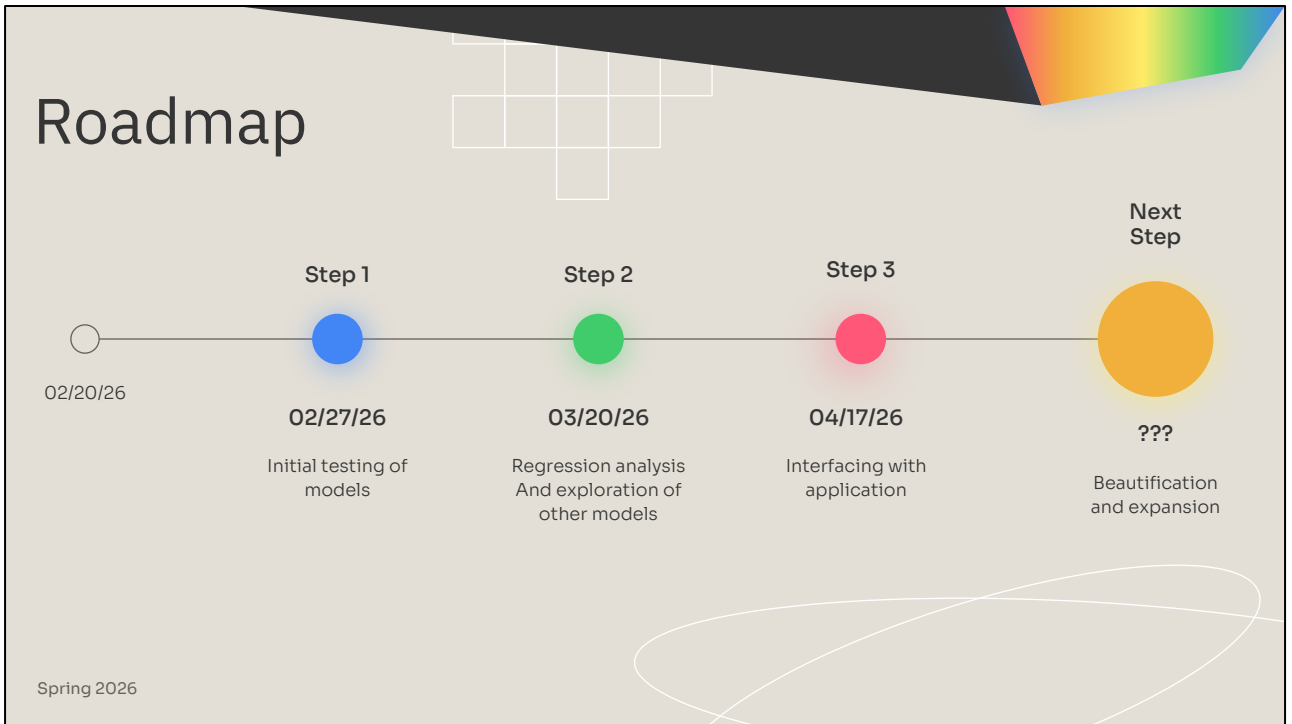
Spring 2026

Michael

Having selected the Random Forest model, we then interfaced it with a Tkinter application designed by Colten.

The user selects values for the seven most important features. Then the app passes those values to the saved model and returns whether the mushroom is likely edible or likely poisonous.

The confidence value comes directly from the model's predicted probability. For Random Forest, that basically reflects how many trees voted for the predicted class. So if almost all the trees vote poisonous, the model reports high confidence.



Colten

This was our Progress over the semester we started on the 27th of february with initial testing, moved to pick important features and exploration of other models on the 20th of march, then began interfacing with tkinter, and had a functional application by the 17th of april, and our next step would be beautification, expansion to other datasets and deployment.

Next steps

05-01-26 Start
Beautification

We're here!

Flesh out current ui look and useability of the application.

06-01-26 Expansion to other
datasets

Expand our model to include other datasets of mushrooms as well.

07-01-26 Final touches and
Deployment

Finish any final touches and reevaluate our goals to decide deployment.

Spring 2026

Colten

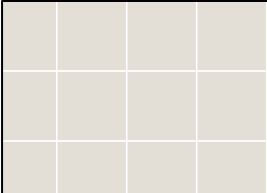
In conclusion, we are very satisfied with the results achieved, and this dataset turned out to be highly separable. Several models performed extremely well, and some achieved 100% accuracy and F1 score.

The most interesting result was that we did not need all 22 features. Using only the top seven features, both Decision Tree and Random Forest reached perfect performance in our tests.

The Decision Tree is the best model for explanation because we can visualize the actual decision process. The Random Forest is probably the stronger application model because it is an ensemble and generally more robust.

Finally, the modular design was a major part of the project. It allowed us test several model variations quickly and compare them in a repeatable way.

Were we to continue developing this further, this is the process we would pursue and approximate timelines, starting with beautification after today, improving the user experience and cleaning things up to look less windows xp, then a month after moving onwards to other datasets (probably sticking to north america), and finally 2 months out looking to finish up any final touches and reevaluate our goals to decide if deployment is prudent.



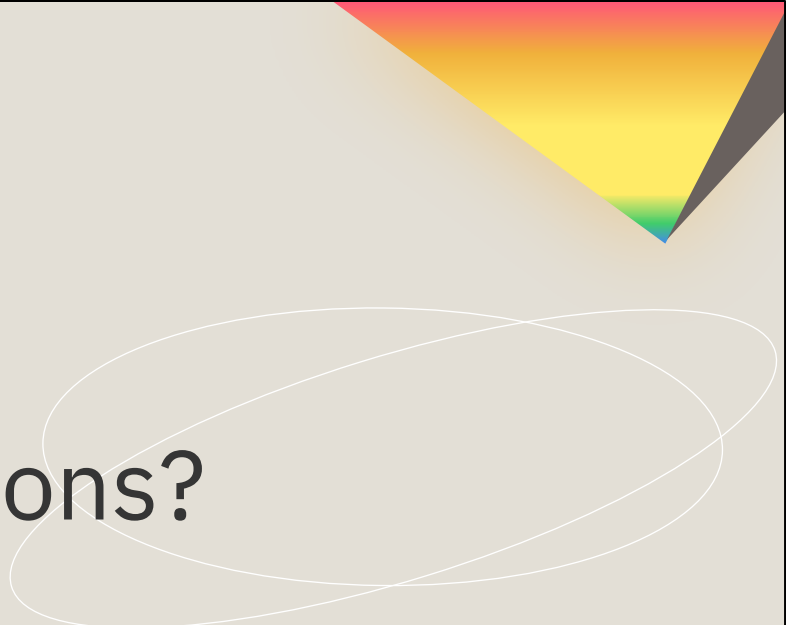
Let's discuss any questions or comments

If something comes up later, contact us:

cjduckworth@mavs.coloradomesa.edu

manuttall@mavs.coloradomesa.edu

cmross@mavs.coloradomesa.edu



Any questions? Ask away!



Thank you